

TALLER DE DISEÑO DE SOFTWARE 1

Programación Web — Paso a Paso

CLASE

05

Funciones

function — parámetros — return — arrow functions

Año 2026

Prof. Ing. Daniel Maldonado

Contenido de esta clase

1. ¿Qué problema resuelve una función?
2. Las 5 partes de una función
3. Invocación — el "Momento 2"
4. ¿Qué hace return?
5. Separación de responsabilidades — calcular vs. mostrar
6. Puente a arrow functions
7. ¿Cuándo uso una u otra?

1 ¿Qué problema resuelve una función?

Arrancamos mostrando el problema SIN funciones.

 **Escribí este código en tu archivo JavaScript y ejecutalo:**

```
let numero1 = 200;
let numero2 = 150;
let suma = numero1 + numero2;
console.log(`La suma 1 es ${suma}`);

let numero3 = 400;
let numero4 = 190;
let suma2 = numero3 + numero4;
console.log(`La suma 2 es ${suma2}`);
```

Cada vez que necesitamos sumar dos números, repetimos la misma lógica (+) con nombres de variables distintos. Si mañana necesitamos sumar un tercer o cuarto par de números, seguimos copiando y pegando el mismo patrón.



Analogía

Una función es como una máquina que armás una sola vez. Si necesitás exprimir naranjas todos los días, no reconstruís el exprimidor cada mañana — lo armás una vez y lo usás todas las veces que haga falta, solo cambiando qué naranja le metés adentro.

Una función es un subprograma que agrupa una tarea específica, para poder reutilizarla las veces que necesitemos sin reescribir la lógica.

2 Las 5 partes de una función

Usamos este ejemplo como referencia:

```
function sumaDeDosNumeros (numero1, numero2)
{
  let auxiliar = 0;
  auxiliar = numero1 + numero2;
  return auxiliar;
}
```

Parte	¿Qué es?	En el ejemplo
function	Palabra reservada que indica que estamos definiendo una función	function
Nombre	Nombre representativo de lo que hace la función	sumaDeDosNumeros
Parámetros	Datos de entrada, entre paréntesis	(numero1, numero2)
Cuerpo	El código que se ejecuta, entre llaves	{ let auxiliar = 0; ... }
return	Lo que la función devuelve como resultado	return auxiliar;

Importante

Definir una función no la ejecuta. Es como escribir la receta de un plato — hasta que alguien no la cocina, no pasa nada. Eso lo vemos en el punto siguiente.

3 Invocación — el "Momento 2"

Hay un momento en que definimos la función, y otro momento distinto en que la invocamos (la usamos).

 **Escribí este código en tu archivo JavaScript y ejecutalo:**

```
// Momento 1 — definición (esto NO ejecuta nada todavía)
function sumaDeDosNumeros(numero1, numero2) {
  let auxiliar = 0;
  auxiliar = numero1 + numero2;
  return auxiliar;
}

// Momento 2 — invocación (acá SÍ se ejecuta la función)
let resultado1 = sumaDeDosNumeros(15, 20);
let resultado2 = sumaDeDosNumeros(200, 400);
let resultado3 = sumaDeDosNumeros(500, 700);

console.log(`El resultado es ${resultado1}`);
console.log(`El resultado es ${resultado2}`);
console.log(`El resultado es ${resultado3}`);
```

La misma función, definida una sola vez, se invoca tres veces con parámetros distintos. Ahí está el ahorro real de código respecto al ejemplo del punto 1.

4 ¿Qué hace return?

Diferencia clave entre una función que devuelve algo y una que solo muestra algo.

✓ Con return — devuelve un valor que se puede guardar y reutilizar

```
function calcularIVA(precio) {  
  let precioFinal = precio + (precio * 21 / 100);  
  return precioFinal;  
}  
  
let total = calcularIVA(1000);  
console.log(total);    // 1210  
console.log(total * 2); // 2420 — puedo seguir operando con él
```

✗ Sin return — solo muestra, no devuelve nada usable

```
function calcularIVASinReturn(precio) {  
  let precioFinal = precio + (precio * 21 / 100);  
  console.log(precioFinal);  
}  
  
let total2 = calcularIVASinReturn(1000);  
console.log(total2);    // undefined  
console.log(total2 * 2); // NaN — no hay nada para operar
```

⚠ Importante

Si una función no tiene return, guardar su resultado en una variable da undefined. Es uno de los errores más comunes al empezar — la función "parece funcionar" porque el console.log de adentro muestra algo, pero no hay nada que devolver hacia afuera.

return no solo sirve para números — también podemos devolver true o false:

```
function esMayorDeEdad(edad) {  
  return edad >= 18;  
}  
  
console.log(esMayorDeEdad(20)); // true  
console.log(esMayorDeEdad(15)); // false
```

5 Calcular vs. mostrar

Regla de oro

Una función debe realizar cálculos, comparaciones o procesos — NO debe mostrar resultados con `console.log()`. Quien invoca la función es responsable de decidir qué hacer con el resultado, incluido mostrarlo.

✗ MAL — la función muestra el resultado, no lo devuelve

```
function calcularIVA(precio) {  
  let precioFinal = precio + (precio * 21 / 100);  
  console.log(`El precio con IVA es: ${precioFinal}`);  
}  
  
calcularIVA(1000);  
// La función "hizo todo" — pero si necesito ese valor  
// para otra cuenta, no puedo usarlo.
```

✓ BIEN — la función calcula y devuelve, la invocación muestra

```
function calcularIVA(precio) {  
  let precioFinal = precio + (precio * 21 / 100);  
  return precioFinal;  
}  
  
let total = calcularIVA(1000);  
console.log(`El precio con IVA es: ${total}`);  
// "total" queda disponible para seguir operando  
// con él si hace falta.
```

El mismo patrón, repetido con más funciones. En las tres, la función solo calcula y devuelve — el `console.log()` vive siempre afuera, en el lugar donde se invoca.

calcularDescuento(precio, porcentaje)

```
function calcularDescuento(precio, porcentaje) {  
  let descuento = precio * porcentaje / 100;  
  let precioFinal = precio - descuento;  
  return precioFinal;  
}  
  
let final = calcularDescuento(20000, 10);  
console.log(`Precio con descuento: ${final}`);
```

calcularPromedio(nota1, nota2, nota3)

```
function calcularPromedio(nota1, nota2, nota3) {  
  let promedio = (nota1 + nota2 + nota3) / 3;  
  return promedio;  
}  
  
let miPromedio = calcularPromedio(8, 6, 9);  
console.log(`Tu promedio es: ${miPromedio}`);
```

calcularArea(base, altura)

```
function calcularArea(base, altura) {  
  let area = base * altura;  
  return area;  
}  
  
let areaRectangulo = calcularArea(5, 3);  
console.log(`El área es: ${areaRectangulo}`);
```

6 Puente a arrow functions

No la presentamos como un concepto nuevo separado — la construimos transformando la función que ya conocemos, paso a paso.

Paso 0 — la función original

```
function sumaDeDosNumeros(numero1, numero2) {  
  return numero1 + numero2;  
}
```

Paso 1 — la guardamos en una constante

```
const sumaDeDosNumeros = function(numero1, numero2) {  
  return numero1 + numero2;  
};
```

Paso 2 — reemplazamos function por => (arrow)

```
const sumaDeDosNumeros = (numero1, numero2) => {  
  return numero1 + numero2;  
};
```

Paso 3 — si el cuerpo es una sola línea con return, se simplifica

```
const sumaDeDosNumeros = (numero1, numero2) => numero1 + numero2;
```

Importante

Mismo comportamiento, mismo resultado al invocarla — `sumaDeDosNumeros(15, 20)` sigue funcionando exactamente igual. Solo cambia la sintaxis de cómo se escribe.

El mismo paso a paso aplicado a otras dos funciones que ya conocemos:

alCuadrado(numero)

```
// Con function:  
function alCuadrado(numero) {  
  return numero * numero;  
}  
  
// Como arrow function:  
const alCuadrado = (numero) => numero * numero;
```

esMayorDeEdad(edad)

```
// Con function:  
function esMayorDeEdad(edad) {  
  return edad >= 18;  
}  
  
// Como arrow function:  
const esMayorDeEdad = (edad) => edad >= 18;
```

7 ¿Cuándo uso una u otra?

- Ambas formas conviven en el código real y vas a ver las dos constantemente.
- Las arrow functions son más comunes hoy en día, sobre todo en callbacks cortos — algo que van a ver pronto con métodos de arrays.
- Por ahora, lo importante es reconocer que son la misma idea con dos sintaxis distintas — no dos temas diferentes.

function	Arrow function
<code>function sumar(a, b) { return a + b; }</code>	<code>const sumar = (a, b) => a + b;</code>

¿Qué viene en la próxima clase?

Vamos a combinar todo lo que aprendimos — condicionales, ciclos y funciones — con arrays y objetos, para empezar a trabajar con listas de datos reales.